# Automated Construction of Embedded Software IP from Legacy Code: Entity Extraction, Interface Refinement, and Multi-Level Specification Verification

CCS Concepts: • **Software and its engineering** → **Software design engineering**; **Formal software verification**; *Reusability*; Software libraries and repositories.

Additional Key Words and Phrases: Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

**ACM Reference Format:**
. 2026. Automated Construction of Embedded Software IP from Legacy Code: Entity Extraction, Interface Refinement, and Multi-Level Specification Verification. 1, 1 (November 2026), 13 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 Introduction

Wang Shuling

## 2 Background and Motivating Example

### 2.1 Software IP Overview

Xu Xiong: Briefly introduce the concept of Software IP (Entity, Interface, and Formal Specification). Do not expand on the Knowledge Model (we will not include this part in this paper).

### 2.2 A Motivating Example

Yang Fanpeng, Ma Xu: (1) Present a real-world aerospace C code snippet. (2) Highlight the engineering pain points: hidden global dependencies, pointer aliases, and floating-point complexities.

## 3 Automated Extraction of Software IP Entities

Extracting a reusable entity from legacy C code requires more than copying a function body. Its implementation may depend on declarations distributed across files, global data accessed through callees, and definitions embedded in macros. Moreover, adapting the function to the software IP invocation convention changes how arguments and results are exchanged. The extraction procedure combines collaborative parsing and code refactoring to collect explicit and implicit dependencies and generate standardized software IP entities. Classification of interface variables is deferred to Section 4.

Author's Contact Information:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to ACM

### 3.1 Overview of the Extraction Pipeline

The procedure takes a C project, a target entry function, and a list of library headers excluded from extraction. It collects dependencies associated with the entry and the functions selected for extraction, then generates an interface header and an source file for each entity. The header contains the context declaration and required type and macro definitions; the implementation provides a void-returning entry procedure that receives a context pointer. The infrastructure header, IP.h, defines the common invocation framework and is not part of any individual entity. Excluded libraries remain external dependencies supplied by the build environment.

Extraction proceeds in two stages. First, compiler-derived information and original source structure are collected into a global dictionary. Second, this dictionary guides dependency reconstruction and call-site rewriting. The resulting context exposes data dependencies that were implicit in the original implementation, without yet assigning input, output, parameter, or state roles to its fields.

### 3.2 Dual-Engine Parsing and Global Dictionary Construction

The extractor uses libclang to obtain types, declarations, symbol references, and call relationships from the compiler's representation of the project. It also uses Tree-sitter to inspect the original source, including macro definitions before expansion. These views serve different purposes: compiler information identifies the entities referenced by an implementation, whereas source-level structure supplies definitions that must be retained or rewritten when reconstructing that implementation. In particular, expanded code alone does not provide the original macro body required for source reconstruction.

The global dictionary comprises function, macro, type, global-variable, and context dictionaries. They record function parameters, return types and source code; original macro definitions; types and aliases; global-variable declarations; and call-site nesting and scope, respectively. Dependencies between these dictionaries guide extraction and refactoring. For types, the extractor recursively follows aliases and nested declarations. Aliases originating from the same declaration are associated with a representative record, allowing the analysis to share the underlying definition while retaining the names needed by the source. For macros, identifiers in the unexpanded body are matched against collected definitions to recover dependencies on other macros, types, and global variables. This supplements compiler-derived information with dependencies needed to reproduce the source-level definitions.

Call-context records serve a different purpose from declaration records: they identify where a call occurs within an expression or control construct. This distinction matters during generation because retaining the required declarations does not determine where a replacement invocation can execute. The next stage therefore uses both the dependency information and the syntactic context of each affected call.

### 3.3 Context-Sensitive Code Refactoring

The generator packages the original parameters, referenced global data, and return-value storage into an IP context structure. The generated entry procedure casts its generic context pointer to the IP context structure type and redirects the corresponding accesses to structure members. Required declarations accompany the generated code. After refactoring, accesses to external variables are expressed through structure members.

Because the entry procedure returns void, a value-producing call inside an expression must be rewritten. The extractor uses a post-order traversal to process nested calls from the inside outward, introduces temporaries for intermediate results, emits the standardized invocations, and replaces the original call expressions with accesses to those results. The placement of these invocations depends on the surrounding construct. For example, a call in a while condition is evaluated at the beginning of a generated while (1) body, followed by a condition check that exits the loop when false. A continue statement therefore reaches the condition computation again. A for loop can similarly retain its initialization and iteration expressions while moving the condition computation and exit check to the beginning of its body.

Macros containing affected calls require corresponding changes. An expression macro can be rewritten as a do–while (0) statement block with an additional result-pointer parameter; its invocation is then emitted separately, and the surrounding expression uses the stored result. These transformations explain why source-level macro structure and call-context information are both needed: the former identifies the definition to rewrite, and the latter determines where the rewritten operation is executed.

Code refactoring aims to preserve the evaluation order and logical behavior of the original program. Extraction results depend on the analyzed project code and compilation configuration. Compilation checks the generated declarations and syntax, but does not establish behavioral equivalence. The extraction results are evaluated in Section 6, while Section 4 refines the entity interface using access information from the original project.

## 4 Refinement of IP Interface

Entity extraction makes the variables used by a software IP explicit, but their C types and declarations do not distinguish configuration parameters, internal state, and data exchanged with the environment. These distinctions require analysis of variable accesses, including side effects introduced by pointer aliases and function calls. We therefore combine bottom-up behavior extraction with system-wide exclusivity checks to classify interface variables as parameters, state, inputs, outputs, or inputs and outputs.

The analysis takes the project source, a target entry function, and manually supplied behavior summaries for functions whose source code is unavailable. It comprises preprocessing, behavior extraction, and interface refinement and classification. The output is a JSON description recording the behavioral category of each interface variable. The analysis uses the original project context, since the target function alone does not establish whether other functions modify a variable.

### 4.1 Bottom-Up Behavior Extraction

Preprocessing constructs an abstract syntax tree and a system-wide call graph, and identifies global variables and hardware addresses referenced by the code. The procedure assumes that the analyzed call graph contains no recursion. Reverse topological ordering ensures that each callee's behavior summary is available before its caller is analyzed. A function behavior summary records read and write sets and an abstract memory-state mapping.

Intraprocedural analysis is partially flow-sensitive: it accounts for statement order when updating pointer mappings and performing write-before-read checks, but its treatment of complex control flow is limited. Aggregate accesses are flattened into atomic variables, such as sensor.param.x, so that fields with different access behaviors can be distinguished. Dynamic array indices can be represented by a wildcard, as in

g_RwState[#]. Abstract memory objects are introduced when needed, and pointer assignments update the relationships between pointers and these objects. This representation allows accesses through aliases to contribute to the read and write sets of the corresponding data, rather than being treated only as accesses to pointer variables.

The analysis also checks whether a read follows an overwrite. If the incoming value has already been overwritten along an execution path, a subsequent read on that path depends on an intermediate result rather than on the initial environment. Such reads can be filtered when the overwrite holds on every path reaching the read. The current write-before-read treatment does not always maintain this distinction across branches and can suppress reads that still depend on incoming values, as discussed below.

At a function call, actual-to-formal parameter mapping and memory-object mapping transfer the callee's read and write effects into the caller's context. Repeating this process in reverse topological order propagates side effects through nested calls. Functions without available source are handled through external behavior summaries, which describe parameter accesses and relevant memory-state changes. For example, the summary for memset records a write to the destination memory. When a callee invokes memset on a field of g_RwState, the write is mapped back to the corresponding object and included in the caller's behavior summary.

### 4.2 Refinement and Classification of Interface Variables

Read and write information alone cannot distinguish a parameter from a runtime input, or internal state from shared data. The classification therefore combines the target function's behavior summary with system-wide write information. Here, reads denote the dependence on incoming values retained by behavior extraction, and the target's summary includes propagated callee effects. The five categories are determined as follows.

Parameters are global variables read by the target but not modified by any function within the analyzed system. They represent configuration values, such as a control gain. State variables are global variables both read and written by the target that satisfy the exclusivity check: no other function is identified as writing them. They represent persistent data maintained by the target, such as an accumulator.

Inputs are variables read but not written by the target, excluding those classified as parameters. Outputs are variables written by the target without a detected dependence on their incoming values; explicit function return values are also outputs. Input–output variables are both read and written but do not satisfy the state-variable exclusivity condition, including shared variables written by other functions. Applying the parameter and state conditions alongside the read and write distinctions separates these categories without requiring additional notation.

The classification depends on the extracted behavior summaries. In particular, the write-before-read filtering can suppress a dependence when a conditional write is followed by a read: if the write is treated as an overwrite for all paths, a state or input–output variable may be classified as an output. This is an under-approximation of the read set, rather than evidence that the incoming value is unnecessary. Classification also depends on the coverage of the analyzed code and the supplied external summaries. Section 6 evaluates interface extraction. Section 5 presents a complementary floating-point verification workflow on independently selected C inputs; it does not consume these interface classifications.

## 5 Specification Generation and Multi-Level Verification

This section presents a verification workflow for floating-point software IP whose guarantees must account for C execution and high-level functional behavior. The workflow combines traditional bounded model checking of the original C program, a CompCert-derived executable semantics, and machine-checked reasoning in Rocq. The levels answer different questions: whether selected implementation properties hold within a stated bound, whether the formal model denotes the intended C source under a fixed semantic configuration, and whether the resulting executions satisfy properties that require induction. None of these results is used as a substitute for the obligations of another level.

### 5.1 Relationship to IP extraction and interface refinement

Sections 3 and 4 address the construction of reusable software IP: they identify entity dependencies, rewrite an implementation to the IP calling convention, and classify interface variables. Section 5 addresses a complementary assurance problem: proving properties of floating-point C programs under explicit source semantics. The verification cases used here are selected independently; they are not outputs of the entity extractor, and the interface classifications from Section 4 are not used to generate their proof preconditions. Accordingly, this paper does not claim that the verification workflow is already an end-to-end stage of the extraction pipeline.

The connection is at the level of the paper's goal. Entity and interface analysis makes component boundaries and dependencies explicit, while the verification workflow supplies semantic and functional evidence for selected floating-point C components. These are complementary parts of trustworthy reuse, evaluated on their respective inputs. A future integration could apply the verification workflow to extracted entities and prove that extraction preserves the legacy behavior; that relation is outside the current verification cases and is not assumed by their proofs.

### 5.2 Framework overview

FIVE organizes verification around one C implementation and a fixed semantic configuration, as shown in Figure 1. The input is a translation unit with its required headers, definitions, and external-call contracts, together with the selected function and observations. These verification inputs are independent of the extraction experiments in Sections 3 and 4. A compilable program must additionally lie within the supported CompCert C dialect; successful compilation by another compiler does not establish this requirement [2].

The implementation-checking branch runs traditional CBMC bounded model checking. The model-generation branch exports CompCert Csyntax and exposes the original Csem/Cexec transition semantics in Rocq. These branches can start independently. Their handoff to the proof stage consists of explicit artifacts: a source-bound model, pending obligations, and, when available, CBMC property descriptions and counterexamples. A successful CBMC run is not a premise that Rocq may assume.

The three stages correspond to `five-cbmc`, `five-align`, and `five-rocq`. The wrapper `five-verify` coordinates their handoffs and accounts for evidence and resources; it is not a fourth proof method. A model-generation request stops at pending goals, while a proof request supplies preconditions and proof terms. Rocq can also accept existing models independently of CBMC. This separation makes an unfinished proof visible instead of treating model export as verification.

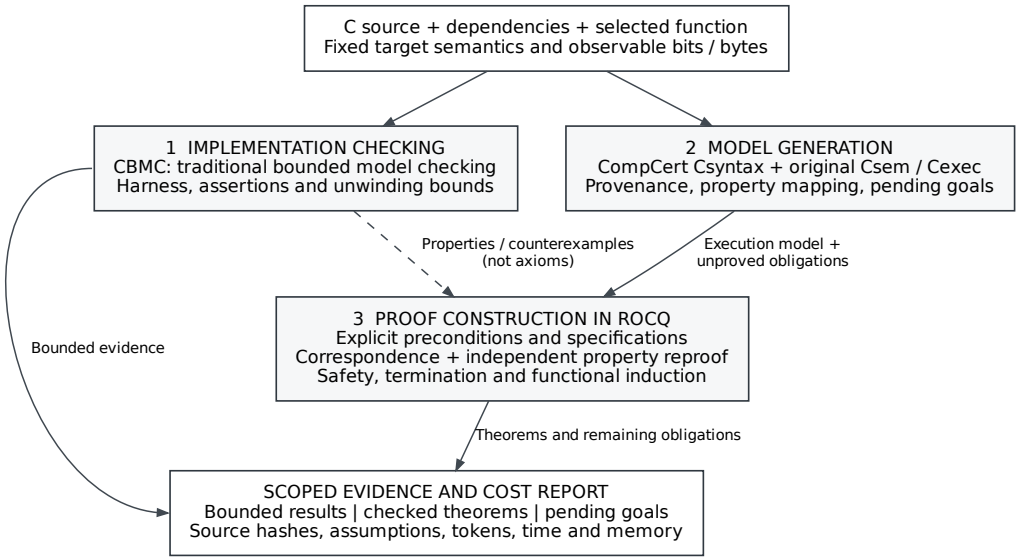



Fig. 1. FIVE verification framework. Solid arrows carry artifacts; the dashed arrow supplies optional CBMC properties and counterexamples for independent analysis. Model generation creates pending goals, not completed proofs. Bounded CBMC evidence and Rocq theorems retain their respective scopes in the final report.

### 5.3 Verification objective and semantic baseline

The objective is to prove functional properties of a floating-point C program while retaining the effects of finite-precision evaluation. A real arithmetic recurrence is not, in general, a sufficient specification of a floating-point filter. Consider the Paparazzi second-order low-pass filter used in our aviation case. Its C implementation computes

$$y = b_0 x + b_1 i_0 + b_0 i_1 - a_0 o_0 - a_1 o_1,$$

where $i_0, i_1$ and $o_0, o_1$ are the stored input and output histories. Writing this recurrence over real numbers hides the rounding performed by each multiplication and by the left-associated additions and subtractions. If $\mathrm{fl}_{32}$ denotes rounding to the selected binary32 format, the corresponding computational structure is

$$\begin{aligned} r_0 &= \mathrm{fl}_{32}(b_0 x), \\ r_1 &= \mathrm{fl}_{32}(r_0 + \mathrm{fl}_{32}(b_1 i_0)), \\ r_2 &= \mathrm{fl}_{32}(r_1 + \mathrm{fl}_{32}(b_0 i_1)), \\ r_3 &= \mathrm{fl}_{32}(r_2 - \mathrm{fl}_{32}(a_0 o_0)), \\ y &= \mathrm{fl}_{32}(r_3 - \mathrm{fl}_{32}(a_1 o_1)). \end{aligned}$$

These equations illustrate the rounding structure for finite operands; the actual model uses CompCert floating-point operations, including its exceptional-value semantics. Reassociating the terms or using a

fused operation can change the output bits. The function then shifts both histories, stores the current input and output, and returns the output. Consequently, an exact functional specification must also account for the updated memory bytes. A different computational structure requires a separate bitwise-equivalence argument, rather than an appeal to algebraic equality over the reals.

The current artifacts use CompCert 3.17 on x86-64 Linux, with little-endian memory, binary32 operations, round-to-nearest ties-to-even and the pinned CompCert NaN policy. Our semantic baseline fixes the CompCert version and target configuration used for each experiment, including architecture, ABI, endianness, floating formats, rounding, and the treatment of exceptional floating-point values. The C translation is obtained from the source with CompCert's C front end; the execution model uses the original CompCert C semantics and executable semantics. It does not replace C execution with a hand-translated arithmetic program. The claims are consequently relative to this formal C semantics and configuration. They do not, by themselves, establish equivalence with an arbitrary GCC build or a particular hardware binary; such a claim would also require a verified or separately validated compilation chain and the corresponding platform assumptions [5].

The model admits the program's machine-level inputs without imposing a numeric filter such as finiteness or non-NaN values. A theorem may state preconditions required by a caller, data structure, or safety property, but those assumptions are recorded with that theorem and are not silently folded into the execution model. The verification target includes observable floating-point results and, when the program uses casts, unions, byte operations, or pointer reinterpretation, the relation between values and the bytes read or written in C memory. IEEE-style floating-point theories make the operation semantics explicit, but a value-level result alone does not establish a byte-level memory property [1].

### 5.4 Why verification is divided into levels

The central design choice is to separate implementation checks, semantic anchoring, and functional reasoning. These tasks require different evidence and need not be discharged by the same proof mechanism. Bit-precise bounded model checking is useful for finding concrete violations and checking local C properties, but its completeness depends on the unwinding bounds. A source-level proof can establish an inductive property for every reachable state, but only if the formal model is tied to the program and the property accurately describes the required behavior. An executable model can expose rounding and memory effects, but execution examples alone do not prove a universal theorem.

This separation is especially relevant to floating-point code. Precise floating-point theories are available to automated reasoning tools; their existence does not imply that a solver will discharge every verification condition produced by a large program. In practice, a condition can combine rounding-sensitive arithmetic with path conditions, array indices, and memory updates. The relevant automation cost therefore has to be measured on the target programs. For CBMC, the evaluation protocol requires verification outcomes together with unwinding bounds, time, and peak memory; a timeout or out-of-memory result is not reported as a proof. Frama-C/WP documents both the precise floating-point model and the fact that such obligations provide limited support for automatic provers. Its Bytes memory model also describes floating-point value-to-integer encoding through a conversion function that the model assumes rather than defines. These are concrete trade-offs for this task, rather than evidence that either framework is generally unsuitable [3].

Table 1. Verification levels and the evidence each level contributes.

| Level | Main mechanism | Suitable obligations | Scope of the result |
|---|---|---|---|
| Implementation | Traditional CBMC bounded model checking | Array and pointer safety, conversions and bit operations, and local assertions over the original C | Complete only for the stated bound or a separately justified complete unwind |
| Model generation | CompCert C front end and original Csem/Cexec semantics | Exported syntax, provenance, pending goals and mapped property statements | Source-bound artifacts; correspondence and program goals remain proof obligations |
| Proof construction | Rocq definitions, lemmas, and induction | Correspondence, CBMC-property reproof, safety, termination and functional invariants | Only completed theorems, under their stated preconditions and environment assumptions |

The required specification also spans different abstraction levels. Contracts and separation-logic assertions are useful for describing function boundaries and memory footprints. For this work, however, the high-level properties include relationships across multiple calls and control cycles. They are naturally expressed with recursive definitions, inductive predicates, and lemmas in a proof assistant. CBMC's ordinary bounded workflow can establish a property up to a chosen loop-unwinding bound; this alone does not imply the property for an arbitrary number of cycles [4]. An inductive invariant or a proof that the bound covers every execution is still required.

Table 1 summarizes how the workflow assigns these obligations. The choice is not based on one tool dominating every dimension. It is based on using CBMC for direct, bit-precise analysis of the C implementation, CompCert for the semantic account of the source, and Rocq for properties whose proof requires induction or reusable functional abstractions.

### 5.5 Level 1: bounded checks of the C implementation

At the first level, CBMC analyzes the selected original C source and its verification entry points using its traditional symbolic-execution and bounded-model-checking workflow. The harness states the input conditions and properties for that verification case; the assertions remain implementation-level C properties. Depending on the selected program and bound, they cover array-index validity, pointer reads and writes, input constraints, bit operations, data conversions, and local functional conditions. CBMC's bit-precise encoding makes it suitable for examining finite-width operations and supported floating-point constructs directly in C [4].

Each result records the source revision, compiler and CBMC options, preconditions, loop-unwinding bounds, assertions, solver result, wall-clock time, and peak memory. A counterexample is retained with its path, input bit patterns and relevant memory effects for comparison with the CompCert model. Reproduction may use a semantic derivation in Rocq; fast direct interpreter evaluation is not required and is not assumed to be available for every model. A successful bounded check is reported as bounded unless the selected unwind is proved to cover every relevant execution. This distinction prevents a finite exploration from being presented as an unbounded safety theorem.

This level is not expected to carry every system-level functional theorem. Its role is to produce direct evidence about concrete C operations and memory accesses, and to discover small counterexamples early. The observed solver cost is an empirical result: the paper reports it per target and property instead of inferring that a floating-point theory is inherently intractable.

### 5.6 Level 2: CompCert model and obligation generation

The second level anchors each selected verification input to a formal C execution model. CompCert's front end automatically translates the C translation unit and its dependencies into its C abstract syntax. Rocq then uses the original CompCert C execution semantics rather than a manually implemented interpreter. This gives the proof a machine-checked account of C expressions, control flow, calls, and memory under a fixed CompCert version and target configuration [5, 6].

The alignment check binds the generated syntax tree and executable model to the selected C files, headers, preprocessing options, and target metadata. Regeneration is compared with the committed model so that a stale or hand-edited export cannot silently become the proof input. This provenance check does not prove the preprocessing and front-end export chain correct; these toolchain components and the chosen external contracts remain explicit trust assumptions. Model state includes the CompCert memory state. Where a property observes memory as bytes, the proof states the representation relation between a C floating-point value, its bit pattern, and the affected byte range; a value-level floating-point theorem is not used as a substitute for this relation.

CBMC results are not imported into Rocq as axioms or trusted certificates. For each selected assertion, this stage records the corresponding proposition as a pending task over the CompCert execution model. The subsequent Rocq proof stage must establish or refute it independently. The reproof uses the same C source, precondition, program point, observable result, and stated bound. If the Rocq theorem proves a stronger unbounded statement, that stronger scope is reported explicitly. A mismatch can reveal an extraction, configuration, or specification error; agreement supplies proof-backed evidence for the selected property under the formal C semantics.

CompCert export binds the formal execution model to the selected C input. It does not assert that this input was produced by the entity-extraction pipeline in Sections 3 and 4; the verification cases are independent of that pipeline.

### 5.7 Explicit safety and termination obligations

The execution domain is separate from the proof precondition. Let $\mathrm{entry}_f(a, m)$ be a function-call state for argument values $a$ and CompCert memory $m$, and let $s \longrightarrow s'$ mean that a Csem transition exists with some trace. The selected function-level error predicate is

$$\mathrm{Error}(s) \equiv \neg\,\mathrm{NormalReturn}(s) \;\wedge\; \neg\exists s'.\, s \longrightarrow s'.$$

Here a normal return has the designated function boundary's stop continuation. This is a semantic no-progress condition, not a claim that the host operating system produces a crash. NaN and infinity are values, not errors by this definition. Exhausting interpreter fuel also does not establish an error. Missing external-call support must be resolved against an explicit environment contract before it is interpreted as program failure.

For every new export, the generator produces separate precondition, obligation and proof-template files. With a still-unspecified predicate $P(a, m)$, the safety target is

$$\forall a, m, s. \quad P(a, m) \wedge \mathrm{entry}_f(a, m) \longrightarrow^* s \;\Rightarrow\; \neg\,\mathrm{Error}(s).$$

The termination target requires accessibility of the successor relation from the entry state, with relation orientation $R(s', s) \equiv s \longrightarrow s'$. Thus every execution branch must be well founded, rather than just one selected evaluation path terminating. Safety and termination are independent obligations: a stuck state can be well founded, and an infinite execution can have progress at every step. Together they exclude both stuck maximal executions and infinite ones, under the stated transition semantics and precondition.

The generator does not select a convenient input domain or insert an assumed theorem. During proof construction, the author supplies $P$, checks that it captures the caller contract and has meaningful inhabitants, and proves the requested goals. A typical safety argument establishes an invariant at entry, proves preservation by every transition, and derives progress or normal return at every reachable state. Induction over reachability then establishes the target without enumerating paths. Termination additionally needs a well-founded argument; the invariant alone does not imply it. The historical cases retain different proof organizations and are not all complete instances of the new generator.

### 5.8 Level 3: proof construction and functional properties in Rocq

At the third level, Rocq proves properties that are awkward to express as bounded assertions over a single C execution. The functional specification is an executable or inductive model whose operations preserve the target-relevant floating-point evaluation order. It may expose a more abstract state than the C heap, but the abstraction is accompanied by an explicit relation to the CompCert state and memory. No alternative real-number expression is assumed to be bitwise equivalent merely because it is algebraically equal.

Proofs are organized compositionally. First, function-level theorems state the transitions and memory effects of individual operations. Next, these theorems are composed to establish a control-cycle property. Finally, an induction over the cycle count proves the desired property for every finite execution length. The induction invariant is written as a Rocq predicate over reachable states; the proof establishes that the initial state satisfies it and that every semantic transition preserves it. This provides an unbounded argument without enumerating paths or selecting a finite number of cycles.

The model does not narrow the execution domain to make the translation easier. Preconditions belong to the theorem being proved, for example a valid caller-owned object or an initialized state required by a function contract. NaNs, infinities, signed zero, subnormal values, and overflow are handled according to the pinned formal semantics wherever that semantics defines the operation. Any additional assumptions needed for a particular functional claim are stated next to the claim.

### 5.9 Why CompCert and Rocq form the proof backbone

The method chooses CompCert and Rocq because the main assurance gap is the connection from a concrete C execution to an inductive functional theorem. CompCert supplies the generated source representation and C execution semantics used in that connection. Rocq supports the definitions and inductive proofs needed for high-level behavior, while retaining kernel checking of the completed terms. CBMC complements

these components by checking bounded, implementation-specific obligations with little model construction overhead.

This division also makes the limits visible. CBMC's bounded result remains bounded unless completeness is shown. A Rocq theorem establishes only its formal proposition, so the precondition and representation relation must match the program. CompCert semantics do not automatically establish the behavior of a binary produced by another compiler. And Rocq's expressive logic does not make floating-point proofs automatic: operation-level rounding facts, bit encodings, and memory lemmas may still require substantial proof effort. The workflow is selected to expose and discharge these distinct obligations, not to claim that one tool eliminates the others' limitations.

Three alternatives clarify the choice. Using CBMC alone retains direct C analysis, but ordinary finite unwinding does not supply an arbitrary-cycle induction proof. Using only a hand-translated Rocq model makes high-level reasoning convenient, but adds a separate translation-correctness problem for every program, including its error and memory behavior. Generating the syntax with CompCert and reusing Csem/Cexec avoids maintaining a second, program-specific implementation of C execution. It does not eliminate source-export provenance checks, external-environment assumptions, or proofs relating any higher-level abstraction to that execution.

Rocq is selected here because CompCert's semantic definitions and supporting proofs are already available in the same logic. This permits direct reuse of its memory and floating-point infrastructure without introducing another cross-assistant semantic translation. This is an integration argument, not a claim that another proof assistant cannot express the properties. The benefit is also not guaranteed lower total cost: interactive proof construction can be expensive, and its token, time and memory costs must be measured alongside the automated checks.

For exact functional specifications, a contract still needs a definition of the required computation. For example, specifying the low-pass filter solely by its real-valued recurrence omits the rounding at each product and accumulation, as well as the precise history updates. A contract may instead refer to the ordered floating-point computation and its memory effects. Choosing a contract framework therefore changes how the proof is organized, but does not in itself remove the need for a precise functional model and its connection to the implementation. Reusable high-level properties are stated separately from this model, so that correctness is not reduced to restating the execution definition.

Frama-C/WP and QCP remain relevant alternatives for contract-driven verification. ACSL and separation-logic annotations can express useful preconditions, postconditions, loop invariants, and memory footprints. For the present goal, those contracts can be used as local interfaces, but they do not remove the need to define the exact floating-point and memory semantics or to prove the relation between the implementation and a high-level model. The WP documentation describes the complexity of precise floating-point obligations and the unspecified float conversion in its Bytes memory model; QCP provides a separation-logic-based symbolic- execution workflow for C verification [3, 7]. For this target, the choice concerns proof organization and semantic infrastructure. Contract-based tools can still contribute useful local interfaces and proof obligations.

### 5.10 Counterexample-guided refinement and evidence

The levels form a feedback process. A CBMC counterexample is checked against its harness and mapped to a CompCert execution under matching inputs, memory and observations. The model-side reproduction is established by execution or a semantic proof; a missing reproduction is reported as pending. If the two differ, the source model, target configuration, or property encoding is investigated before the functional theorem is trusted. If they agree and expose a defect, the implementation or its stated precondition is corrected. If a Rocq proof fails, the resulting unsolved obligation is used to refine the invariant, supporting lemma, or specification; it is not discharged by weakening the claim without recording the change.

The reporting protocol requires, for each case, the C and generated-model hashes, the CompCert configuration, the CBMC properties and bounds, the Rocq theorem statements and assumptions, and actual input/output tokens, wall time, tool CPU time and observed peak process memory. Failed attempts remain in the record; missing measurements are marked incomplete, and shared events are deduplicated rather than charged repeatedly. Repository editing and paper compilation are not verification-stage measurements. An end-to-end claim is made only when the relevant implementation checks, independent property proofs, and semantic correspondence obligations agree. Tests and bounded checks remain useful evidence, while an arbitrary-cycle functional claim rests on the Rocq induction and its proved link to CompCert execution.

## 6 Empirical Evaluation

Here we focus on real-world application and engineering value rather than standard benchmark metrics.

### 6.1 Experimental Setup and Case Description

Yang Fanpeng: Introduce the real-world aerospace software assets (e.g., satellite sun-search control system or flight control software).

### 6.2 RQ1: Effectiveness of Entity and Interface Extraction

Ma Xu

### 6.3 RQ2: Verification Effectiveness and Cost

RQ2 asks which obligations are discharged at each level and what resources the verification requires on the selected aerospace programs. For every CBMC property, we record its source location, precondition, unwind bound, result (proved within bound, counterexample, timeout, or resource failure), wall-clock time, and peak memory under a fixed tool configuration. We report the number of discharged properties divided by the number attempted, while listing timeouts and incomplete unwinds separately; neither is counted as a proof. These measurements provide the evidence for claims about CBMC's automation cost on these programs.

For the CompCert and Rocq levels, the evaluation protocol records verification-source hashes, target metadata, regeneration and semantic-alignment results, theorem statements, assumptions, kernel-check status, input/output token usage, wall time, tool CPU time and observed peak memory. A generated obligation is recorded separately from a completed proof. Cached input and reasoning output are subsets of their respective token totals, not additional charges. The extraction/interface cases from RQ1 and the verification

cases from RQ2 are reported as separate populations. We classify properties by implementation safety, local functional behavior, memory-byte correspondence, cross-function behavior, and cycle invariants. This makes it possible to see which obligations require each level and whether any claim depends on a narrower input domain or a finite bound.

No aggregate success rate or resource figure is inferred from a subset of cases. The quantitative results for RQ2 must be filled from the recorded runs after all selected cases have been evaluated; this draft therefore makes no empirical claim about solver scalability.

### 6.4 Others??

## 7 Related Work

## 8 Discussion and Lessons Learned

## 9 Conclusion

## References

[1] Martin Brain, Cesare Tinelli, Philipp Rümmer, and Thomas Wahl. 2015. An Automatable Formal Semantics for IEEE-754 Floating-Point Arithmetic. In *2015 IEEE 22nd Symposium on Computer Arithmetic*. IEEE, 160–167. doi:10.1109/ARITH.2015.26

[2] CompCert Team. 2026. *The CompCert C Language*. Inria. CompCert manual, Chapter 5; accessed November 3, 2026. https://compcert.org/man/manual005.html

[3] Frama-C Team. 2026. *WP Plug-in Manual*. CEA LIST and Inria. Frama-C 33.0 Arsenic; accessed November 3, 2026. https://www.frama-c.com/download/frama-c-wp-manual.pdf

[4] Daniel Kroening and Michael Tautschnig. 2014. CBMC – C Bounded Model Checker. In *Tools and Algorithms for the Construction and Analysis of Systems (Lecture Notes in Computer Science, Vol. 8413)*. Springer, 389–391.

[5] Xavier Leroy. 2009. Formal Verification of a Realistic Compiler. *Commun. ACM* 52, 7 (2009), 107–115. doi:10.1145/1538788.1538814

[6] Tahina Ramananandro, Paul Mountcastle, Benoît Meister, and Richard Lethin. 2016. A Unified Coq Framework for Verifying C Programs with Floating-Point Computations. In *Proceedings of the 5th ACM SIGPLAN Conference on Certified Programs and Proofs*. ACM, 15–26. doi:10.1145/2854065.2854066

[7] Xiwei Wu, Yueyang Feng, Xiaoyang Lu, Tianchuan Lin, Kan Liu, Zhiyi Wang, Shushu Wu, Lihan Xie, Chengxi Yang, Hongyi Zhong, Zihan Zhang, Juanru Li, Naijun Zhan, Zhenjiang Hu, and Qinxiang Cao. 2026. QCP: A Practical Separation Logic-Based C Program Verification Tool. In *Theoretical Aspects of Software Engineering (Lecture Notes in Computer Science, Vol. 16697)*. Springer, 157–173. doi:10.1007/978-3-032-30693-7_10

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009